Розпочато	вівторок 18 березня 2025 10:45 AM
Стан	Завершено
Завершено	вівторок 18 березня 2025 11:57 AM
Витрачено часу	1 година 12 хв

Питання 1

Завершено

Макс. оцінка до 4,0

Встановіть відповідність

Метод shutdown() -	призначений для завершення завантаження задач в пул потоків
Метод-фабрика -	призначений для створенння пулу потоків
Метод compute() -	призначений для виконання обчислень ForkJoinTask задачі

Питання 2

Завершено

Макс. оцінка до 4,0

Чим відрізнаяється ForkJoinPool від звичайного пулу?

- [x] a. Управлінням чергами задач до потоків, що включає крадіжку задачі з черги іншого потоку у випадку, коли черга задач до потоку порожня
- [ ] b. Неможливо завантажити задачі Runnable та/або Callable
- [ ] c. Результат задачі повертається у вигляді об'єкту Future
- [x] d. Можливістю рекурсивного завантаження задач
- [ ] e. Управлінням чергами задач до потоків, що включає упорядкування за їх складністю
- [ ] f. Управлінням чергами задач до потоків, що включає крадіжку задачі з черги іншого потоку у випадку, коли черга задач менша за черги до інших потоків
- [ ] g. Управлінням чергами задач до потоків, що включає крадіжку задачі з іншого потоку, якщо час її очікування більший за очікування інших задач.
- [ ] h. Не потребує завершення методом shutdown()

Питання 3

Завершено

Макс. оцінка до 2,0

Які ключові слова використовуються для управління java-потоками?

- a. thread, wait, notify
- b. volatile, synchronized
- c. volatile
- d. run, lock, synchronized
- e. synchronized

Питання 4

Завершено

Макс. оцінка до 3,0

Які проблемні ситуації можуть виникнути при реалізації багатопоточної програми?

- [x] a. не завершені обчислення
- [x] b. взаємне блокування потоків
- [x] c. голодування потоків
- [x] d. переривання виконання
- [x] e. лайвлок
- [x] f. некоректні обчислення через несинхронізований доступ до спільної пам'яті
- [ ] g. асинхронне виконання
- [x] h. повільне виконання через синхронізовані обчислення

Питання 5

Завершено

Макс. оцінка до 4,0

Чим відрізняється пул потоків від масиву потоків?

- [ ] a. Тільки типом створюваних потоків
- [ ] b. Пул потоків може прийняти на обчислення Callable завдання, а масив потоків - тільки Runnable завдання
- [x] c. Пул потоків буде знаходитись у стані очікування нових завдань доки не буде виконано завершення робти пулу спеціально для цього призначеним методом, а в масиві потоків завершення роботи потоку відбувається для кожного потоку окремо.
- [ ] d. нічим
- [ ] e. Кількістю завдань, що можуть обчислюватись фізично одночасно, при однаковій кількості потоків в пулі та в масиві
- [ ] f. Завдання, що надходить, в пулі обробляється його потоками одночасно, а в масиві - надходить на виконання до одного з потоків.
- [x] g. У пулі потоків використовується повторне використання потоків для обчислення задач та управління чергами задач, а масив потоків використовує потік тільки одни раз
- [ ] h. Кількістю завдань, що можуть бути запущені на паралельне виконання, при однаковій кількості потоків в пулі та в масиві

Питання 6

Завершено

Макс. оцінка до 4,0

Поставте у відповідність кожному терміну його пояснення.

Data Race	Несинхронізований доступ потоків до спільних даних, хоча б один з яких виконує модифікацію даних.
Memory Consistency Errors	Несинхронізований доступ потоків до спільних даних, хоча б один з яких виконує модифікацію даних.
Race Condition	Порядок виконуваних дій впливає на результат асинхронних обчислень

Питання 7

Завершено

Макс. оцінка до 10,0

Напишіть фрагмент коду для пулу 8 потоків, що виконують паралельне обчислення суми 1000000 елементів масиву дійсних чисел, розділяючи обчислення 100 задач.

```
import java.util.concurrent.*;

public class ParallelSum {
    private static final int ARRAY_SIZE = 1_000_000;
    private static final int NUM_TASKS = 100;
    private static final int NUM_THREADS = 8;

    public static void main(String[] args) throws InterruptedException, ExecutionException {
        double[] array = new double[ARRAY_SIZE];
        for (int i = 0; i < ARRAY_SIZE; i++) {
            array[i] = Math.random();
        }

        ExecutorService pool = Executors.newFixedThreadPool(NUM_THREADS);
        Future<Double>[] results = new Future[NUM_TASKS];

        int chunkSize = ARRAY_SIZE / NUM_TASKS;
        for (int i = 0; i < NUM_TASKS; i++) {
            final int start = i * chunkSize;
            final int end = (i == NUM_TASKS - 1) ? ARRAY_SIZE : start + chunkSize;
            results[i] = pool.submit(() -> {
                double sum = 0;
                for (int j = start; j < end; j++) {
                    sum += array[j];
                }
                return sum;
            });
        }

        double totalSum = 0;
        for (Future<Double> result : results) {
            totalSum += result.get();
        }

        pool.shutdown();
        System.out.println("Total sum: " + totalSum);
    }
}
```

Питання 8

Завершено

Макс. оцінка до 3,0

Для яких цілей використовують guarded-блоки в паралельних програмах?

- a. Призупинки виконання обчислень на певний інтервал часу
- b. Призупинки виконання обчислень до виконання певної умови
- c. Повідомлення про завершення обчислень
- d. Переривання обчислень при виконанні певної умови
- e. Очікування повідомлення про завершення обчислень

Питання 9

Завершено

Макс. оцінка до 18,0

Напишіть підзадачі-«автівки», які виконують метод go() з затримкою 2 мілісекунд за умови, що у підзадачі- «світлофор» встановлений сигнал «зелене світло», і знову повторюють цю дію через 400 мілісекунд. «Світлофор» змінює сигнали «зелене» «жовте» «червоне» «жовте» (і знову «зелене») циклічно з затримками 70, 10, 40, 10 мілісекунд відповідно. При зеленому світлі автівки проїжджають по одній. Створіть 100 «автівок» та один «світлофор» і запустіть їх на виконання. Завершити роботу потоків після проїзду 10000 автівок через світлофор.

```
import java.util.concurrent.atomic.AtomicInteger;

public class Main {

    private static final int NUM_CARS = 100;
    private static final int TOTAL_CARS = 10;

    private static String lightSignal = "green";
    private static final Object lock = new Object();

    private static AtomicInteger carsPassed = new AtomicInteger(0);

    private static boolean finished = false;

    public static void main(String[] args) throws InterruptedException {
        Thread trafficLightThread = new Thread(new TrafficLight());
        trafficLightThread.start();

        for (int i = 0; i < NUM_CARS; i++) {
            new Thread(new Car()).start();
        }

        trafficLightThread.join();

        finished = true;
    }

    static class TrafficLight implements Runnable {
        @Override
        public void run() {
            try {

                int sleep_time = 0;

                while (carsPassed.get() < TOTAL_CARS) {
                    synchronized (lock) {
                        lightSignal = "green";
                    }
                    Thread.sleep(70);

                    synchronized (lock) {
                        lightSignal = "yellow";
                    }
                    Thread.sleep(10);

                    synchronized (lock) {
                        lightSignal = "red";
                    }
                    Thread.sleep(40);

                    synchronized (lock) {
                        lightSignal = "yellow";
                    }
                    Thread.sleep(10);
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    // Потік автівки
    static class Car implements Runnable {

        public void go() throws InterruptedException {
            carsPassed.incrementAndGet();
        }

        @Override
        public void run() {
            try {
                while (carsPassed.get() < TOTAL_CARS && !finished) {
                    synchronized (lock) {
```

```
                        if ("green".equals(lightSignal)) {
                            Thread.sleep(2);
                            go();
                            Thread.sleep(400);
                        }
                    }
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Питання 10

Завершено

Макс. оцінка до 3,0

У чому перевага використання інтерфейсу Callable?

- a. повертає значення будь-якого типу
- b. повертає значення об'єктного типу
- c. повертає значення типу Future<V>
- d. більш швидкий
- e. об'єкти цього типу можна використовувати у конструкторі класу Thread

Питання 11

Завершено

Макс. оцінка до 2,0

Які алгоритми НЕ можна розпаралелити?

- a. Які не можна розбити на підзадачі, що можуть виконуватись одночасно
- b. Алгоритми сортування
- c. Які не можна розбити на незалежні підзадачі
- d. Алгоритми неполіноміальної складності
- e. Алгоритми на графах

Питання 12

Завершено

Макс. оцінка до 1,0

Чи може об’єкт бути блокованим?

Виберіть одну відповідь:

- Правильно
- Неправильно

Питання 13

Завершено

Макс. оцінка до 5,0

Виправіть помилки в наступному фрагменті коду:

```
public sync send(int i, int j, double money){
if (accounts[i] < amount)
  await;
accounts[i] —= money;
accounts[j] += money;
signalAll;
}
```

```
public sync send(int i, int j, double money){
if (accounts[i] < amount)
  wait();
accounts[i] -= money;
accounts[j] += money;
notifyAll();
}
```

Питання 14

Завершено

Макс. оцінка до 3,0

За яких умов потік переходить у стан блокований?

- a. За умови, що він намагається блокувати інший потік
- b. За умови, що він намагається блокувати об'єкт, який є заблокованим іншим потоком
- c. За умови, що він призупинив свою роботу
- d. За умови, що він намагається виконати синхронізований метод, або синхронізований блок, або заблокований локером код
- e. За умови, що виникло переривання його роботи

Питання 15

Завершено

Макс. оцінка до 1,0

Чи можна в синхронізованому методі викликати несинхронізований

Виберіть одну відповідь:

- Правильно
- Неправильно

Питання 16

Завершено

Макс. оцінка до 8,0

Напишіть фрагмент коду, в якому створюються та запускаються на виконання потоки A,B,C. Потоки A і B виконують створення та додавання об’єктів в буфер обмеженої довжини, а потік C – їх вилучення. Використовуйте тип Object для об'єктів, які створюються, додаються та вилучаються.

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class ThreadBufferExample {
    private static final int BUFFER_CAPACITY = 10;
    private static final int NUM_ITEMS = 20;

    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<Object> buffer = new ArrayBlockingQueue<>(BUFFER_CAPACITY);

        Thread threadA = new Thread(new Producer(buffer), "Потік A");
        Thread threadB = new Thread(new Producer(buffer), "Потік B");
        Thread threadC = new Thread(new Consumer(buffer), "Потік C");

        threadA.start();
        threadB.start();
        threadC.start();

        threadA.join();
        threadB.join();
        threadC.join();
    }

    static class Producer implements Runnable {
        private final BlockingQueue<Object> buffer;

        public Producer(BlockingQueue<Object> buffer) {
            this.buffer = buffer;
        }

        @Override
        public void run() {
            try {
                for (int i = 0; i < NUM_ITEMS / 2; i++) {
                    Object item = new Object();
                    buffer.put(item);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
```

```
    }

    static class Consumer implements Runnable {
        private final BlockingQueue<Object> buffer;

        public Consumer(BlockingQueue<Object> buffer) {
            this.buffer = buffer;
        }

        @Override
        public void run() {
            try {
                for (int i = 0; i < NUM_ITEMS; i++) {
                    Object item = buffer.take();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
```

Питання 17

Завершено

Макс. оцінка до 4,0

Яка різниця між методом tryLock() і lock() інтерфейсу Lock?

- [x] a. tryLock() - повертає результат успішного/неуспішного захоплення локера, а lock() - не надає жодної інформації про успішність блокування
- [x] b. tryLock() - захоплює локер, якщо він незаблокований, а lock() - очікує звільнення локера
- [ ] c. У разі невдалого захоплення локера lock() - не блокує потік, а tryLock() - блокує потік
- [ ] d. lock() - надає можливість повторного захоплення локера , а tryLock() - забороняє таку можливість
- [x] e. У разі невдалого захоплення локера lock() не дозволяє перехід на наступну інструкцію програми , а tryLock() - дозволяє

Питання 18

Завершено

Макс. оцінка до 8,0

Напишіть фрагмент коду програми, в якому N потоків виконують 1000 ітерацій, в кожній з яких створюють об'єкт String з випадковим набором символів та одразу додають його у спільну для всіх потоків колекцією об'єктів String.

```
import java.util.List;
import java.util.Random;
import java.util.concurrent.CopyOnWriteArrayList;

public class RandomStringThreads {
    private static final int N = 10;
    private static final int ITERATIONS = 1000;

    public static void main(String[] args) throws InterruptedException {
        List<String> sharedList = new CopyOnWriteArrayList<>();

        Thread[] threads = new Thread[N];
        for (int i = 0; i < N; i++) {
            threads[i] = new Thread(new StringGenerator(sharedList));
            threads[i].start();
        }

        for (int i = 0; i < N; i++) {
            threads[i].join();
        }
    }

    static class StringGenerator implements Runnable {
        private final List<String> sharedList;
        private final Random random = new Random();

        public StringGenerator(List<String> sharedList) {
            this.sharedList = sharedList;
        }

        @Override
        public void run() {
            for (int i = 0; i < ITERATIONS; i++) {
                String randomString = generateRandomString(10);
                sharedList.add(randomString);
            }
        }

        private String generateRandomString(int length) {
            StringBuilder sb = new StringBuilder(length);
            for (int i = 0; i < length; i++) {
                char c = (char) (random.nextInt(26) + 'a');
```

```
                sb.append(c);
            }
            return sb.toString();
        }
    }
}
```

Питання 19

Завершено

Макс. оцінка до 12,0

Напишіть код, в якому:
– потік A з затримкою 100 мілісекунд переключається зі стану ”r” у стан “w” і навпаки,
- потік B очікує стану потоку A ”r”, виводить на консоль зворотний відлік мілісекунд від 100 та призупиняє свою дію, як тільки потік A переключено у стан “w”.

```
public class ThreadSwitchingExample {
    private static boolean isInRState = true;  // Стан потоку A (r = read, w = write)
    private static final Object lock = new Object();

    public static void main(String[] args) throws InterruptedException {
        // Потік A
        Thread threadA = new Thread(() -> {
            try {
                while (true) {
                    synchronized (lock) {
                        isInRState = !isInRState;
                        lock.notify();
                    }
                    Thread.sleep(100);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Потік B
        Thread threadB = new Thread(() -> {
            try {
                while (true) {
                    synchronized (lock) {
                        while (!isInRState) {
                            lock.wait();
                        }

                        for (int i = 100; i > 0; i--) {
                            Thread.sleep(1);
                        }
                    }
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
```

```
        threadA.start();
        threadB.start();

        threadA.join();
        threadB.join();
    }
}
```

Питання 20

Завершено

Макс. оцінка до 4,0

Поставте у відповідність типи, визначені бібліотекою java.util.concurrent, їх узагальненим назвам

Синхронізовані типи	BlockingQueue, ConcurrentMap
Атомік типи	AtomicInteger, AtomicBoolean, AtomicReference

Питання 21

Завершено

Макс. оцінка до 3,0

З яких елементів складається guarded-блок?

- a. if(!condition) (wait();}
- b. wait(); notify();
- c. if(condition) guard()
- d. while(!condition) (wait();}
- e. await(), wait(), notify(), signal()

Питання 22

Завершено

Макс. оцінка до 1,0

Чи можна в несинхронізованому методі викликати синхронізований

Виберіть одну відповідь:

- Правильно
- Неправильно

Питання 23

Завершено

Макс. оцінка до 2,0

Які стани потоку визначені документацією мови Java?

- a. RUNNABLE, WAITING, TERMINATED
- b. NEW, RUNNABLE, SUSPENDED, DIED
- c. NEW, RUNNABLE, WAITING, STOPED
- d. NEW, RUNNABLE, TIMED_WAITING, WAITING, TERMINATED
- e. CREATED, RUNNABLE, SUSPENDED, TERMINATED

Питання 24

Завершено

Макс. оцінка до 2,0

Чи може потік декілька разів захопити один і той самий локер?

Виберіть одну відповідь:

- Правильно
- Неправильно